Soluciones explicadas

Cómo leer las soluciones

Cada solución trae cuatro cosas: el código, el resultado esperado, por qué funciona y la trampa en la que suele caer la gente, que es justo lo que buscan los entrevistadores.

- **Resultados comprobados:** todas las consultas (queries) SQL, el código de pandas y los cálculos de estadística se han ejecutado sobre TiendaDemo en PostgreSQL 16 y Python. Si tu resultado no coincide, el error está en tu solución.
- **Hay más de una forma correcta:** si tu código es distinto pero da el mismo resultado, vale. Compara igualmente el razonamiento.
- **Las preguntas abiertas** (negocio, inglés) traen una respuesta modelo. No la memorices: quédate con la estructura.

Bloque 1 · SQL

SQL-1 · Ingresos (revenue) totales

```
SELECT SUM(oi.quantity * oi.unit_price) AS ingresos
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'delivered';
```

Resultado: 2.792,50 €.

Explicación: los importes están en las líneas (order_items) y el estado en los pedidos (orders), así que hay que unir (join) las dos tablas. El importe (amount) de cada línea es cantidad × precio cobrado.

Trampa típica: sin el filtro de estado salen 3.182,50 €, un 14 % más, porque sumas pedidos cancelados y devueltos que no se cobraron. En una entrevista, pregunta siempre qué cuenta como venta antes de empezar.

SQL-2 · Actividad mensual

```
SELECT DATE_TRUNC('month', order_date)::date AS mes,
       COUNT(*)                    AS pedidos,
       COUNT(DISTINCT customer_id) AS clientes
FROM orders
WHERE status = 'delivered'
GROUP BY 1
ORDER BY 1;
```

mes	pedidos	clientes
2026-01	3	2
2026-02	3	3
2026-03	6	6
2026-04	5	5
2026-05	5	5

Explicación: `DATE_TRUNC('month', fecha)` lleva cada fecha al día 1 de su mes, así que sirve para agrupar por mes. `COUNT(DISTINCT customer_id)` cuenta a cada cliente una sola vez aunque compre varias veces.

Trampa típica: `COUNT(customer_id)` sin DISTINCT cuenta pedidos, no clientes: en enero daría 3 en vez de 2.

SQL-3 · Clientes sin compras

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Resultado: Hugo Martín (customer_id 11).

Explicación: el LEFT JOIN mantiene a todos los clientes aunque no tengan pedidos; en esos casos las columnas de orders salen a NULL, y eso es lo que filtras. Igual de válido: `WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)`.

Trampa típica: con un JOIN normal los clientes sin pedidos desaparecen y la consulta (query) no devuelve nada. Pregunta de seguimiento (follow-up question) habitual: «¿y los que solo tienen pedidos cancelados?». Respuesta: añade `AND o.status = 'delivered'`

dentro del ON, no en el WHERE.

SQL-4 · Clientes por ciudad

```
SELECT COALESCE(INITCAP(TRIM(city)), 'Desconocida') AS ciudad,
       COUNT(*) AS clientes
FROM customers
GROUP BY 1
ORDER BY clientes DESC, ciudad;
```

Resultado: Madrid 4, Barcelona 3, Valencia 2, Bilbao 1, Desconocida 1 y Sevilla 1.

Explicación: TRIM quita los espacios sobrantes, INITCAP pone mayúscula inicial y COALESCE sustituye el nulo. Sin limpiar, «Madrid» y « madrid» saldrían como ciudades distintas.

Para destacar en la entrevista: Madrid suma 4 cuentas pero son 3 personas, porque Ana tiene dos cuentas (SQL-5). Señalarlo sin que te lo pregunten suma puntos.

SQL-5 · Emails duplicados

```
SELECT email,
       COUNT(*) AS cuentas,
       STRING_AGG(customer_id::text, ', ' ORDER BY customer_id) AS ids
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Resultado: ana@mail.com, 2 cuentas, ids 1 y 10.

Explicación: agrupas por la columna que debería ser única y te quedas con los grupos de más de una fila. HAVING filtra después de agrupar; WHERE, antes. Es una pregunta clásica para comprobar que conoces la diferencia.

SQL-6 · Ticket medio (average order value, AOV) por canal

```
WITH totales AS (
  SELECT o.order_id, o.customer_id,
         SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.order_id, o.customer_id
)
SELECT c.channel,
       COUNT(*)               AS pedidos,
       ROUND(AVG(t.total), 2) AS ticket_medio
FROM totales t
JOIN customers c ON c.customer_id = t.customer_id
GROUP BY c.channel
ORDER BY ticket_medio DESC;
```

canal	pedidos	ticket medio (€)
ads	8	138,44
referral	5	125,00
organic	9	117,78

Explicación: primero calculas el total de cada pedido y después haces la media (mean) de esos totales por canal.

Trampa típica: si haces AVG directamente sobre las líneas, calculas la media por línea y no por pedido: organic saldría 81,54 €. Pregúntate siempre «media de qué».

SQL-7 · Producto estrella de cada categoría

```
WITH ventas AS (
  SELECT p.category, p.product_name,
         SUM(oi.quantity * oi.unit_price) AS ingresos
  FROM order_items oi
  JOIN orders o   ON o.order_id = oi.order_id
  JOIN products p ON p.product_id = oi.product_id
  WHERE o.status = 'delivered'
  GROUP BY p.category, p.product_name
),
ranking AS (
  SELECT *, RANK() OVER (PARTITION BY category ORDER BY ingresos DESC) AS
puesto
  FROM ventas
)
SELECT category, product_name, ingresos
FROM ranking
WHERE puesto = 1
ORDER BY category;
```

Resultado: Audio: Auriculares (300,00 €); Monitores: Monitor 27″ (1.462,50 €); Periféricos: Teclado mecánico (392,00 €).

Explicación: es el patrón «top N por grupo», el más preguntado en entrevistas. Numeras dentro de cada categoría con una función de ventana (window function) y luego filtras. Hacen falta dos pasos porque el WHERE no puede usar una función de ventana del mismo SELECT.

Matiz: RANK da el mismo puesto a los empates y saldrían los dos; ROW_NUMBER elegiría uno al azar. Menciónalo.

SQL-8 · Crecimiento mensual (month-over-month growth)

```
WITH mensual AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
         SUM(oi.quantity * oi.unit_price) AS ingresos
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY 1
)
SELECT mes,
       ingresos,
       LAG(ingresos) OVER (ORDER BY mes) AS mes_anterior,
       ROUND(100.0 * (ingresos - LAG(ingresos) OVER (ORDER BY mes))
             / LAG(ingresos) OVER (ORDER BY mes), 1) AS crecimiento_pct
FROM mensual
ORDER BY mes;
```

mes	ingresos (€)	crecimiento
2026-01	420,00	—
2026-02	217,00	−48,3 %
2026-03	890,00	+310,1 %
2026-04	493,00	−44,6 %
2026-05	772,50	+56,7 %

Explicación: LAG trae el valor de la fila anterior según el orden indicado. El 100.0 con decimal evita que algunos motores hagan división entera.

Para destacar: con tan pocos pedidos, los porcentajes bailan muchísimo. Con volúmenes pequeños, enséñalos junto a los valores absolutos.

SQL-9 · Ingresos acumulados (running total)

```
WITH mensual AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
         SUM(oi.quantity * oi.unit_price) AS ingresos
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY 1
)
SELECT mes, ingresos,
       SUM(ingresos) OVER (ORDER BY mes) AS acumulado
FROM mensual
ORDER BY mes;
```

Resultado (acumulado): 420,00 → 637,00 → 1.527,00 → 2.020,00 → 2.792,50 €.

Explicación: `SUM(...) OVER (ORDER BY mes)` suma desde la primera fila hasta la actual. El último valor coincide con SQL-1: es una buena comprobación de que no te has dejado nada. Si hubiera varios años, añadirías `PARTITION BY` año para reiniciar el acumulado cada enero.

SQL-10 · Tiempo hasta la segunda compra

```
WITH compras AS (
  SELECT customer_id, order_date,
         LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
AS siguiente,
         ROW_NUMBER()     OVER (PARTITION BY customer_id ORDER BY order_date)
AS n
  FROM orders
  WHERE status = 'delivered'
)
SELECT customer_id,
       order_date             AS primera_compra,
       siguiente              AS segunda_compra,
       siguiente - order_date AS dias
FROM compras
WHERE n = 1 AND siguiente IS NOT NULL
ORDER BY dias;
```

Resultado: 7 clientes han repetido, con 20, 24, 26, 35, 38, 57 y 67 días. La media es de 38,1 días (envuelve la consulta y calcula `AVG(dias)`).

Explicación: LEAD trae la fecha de la siguiente compra del mismo cliente gracias a `PARTITION BY customer_id`. Te quedas con la primera compra (n = 1) que tiene siguiente. En PostgreSQL, restar dos fechas da directamente días.

Para destacar: si la segunda compra llega de media a los 38 días, tiene sentido un email de recordatorio hacia el día 30. Convertir un número en una acción es lo que distingue a un buen analista.

SQL-11 · Concentración de ingresos

```
WITH por_cliente AS (
  SELECT o.customer_id, SUM(oi.quantity * oi.unit_price) AS ingresos
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.customer_id
),
ranking AS (
  SELECT customer_id, ingresos,
         ROW_NUMBER() OVER (ORDER BY ingresos DESC) AS puesto,
         SUM(ingresos) OVER ()                     AS total
  FROM por_cliente
)
SELECT ROUND(100.0 * SUM(ingresos) / MAX(total), 1) AS pct_top3
FROM ranking
WHERE puesto <= 3;
```

Resultado: 51,6 %. Los tres primeros (Luis 617,50 €, Sara 457,00 € y Diego 366,00 €) suman 1.440,50 de 2.792,50 €.

Explicación: `SUM(...) OVER ()` con los paréntesis vacíos calcula el total de todas las filas y lo repite en cada una, así que puedes dividir sin una segunda consulta.

Lectura: tres clientes de once aportan más de la mitad. Perder a uno de ellos se notaría mucho.

SQL-12 · Embudo de conversión (conversion funnel)

```
WITH pasos AS (
  SELECT event_type, COUNT(DISTINCT user_id) AS usuarios
  FROM events
  GROUP BY event_type
),
orden AS (
  SELECT * FROM (VALUES ('view_product', 1), ('add_to_cart', 2),
                        ('checkout', 3), ('purchase', 4)) AS v(event_type,
paso)
)
SELECT o.paso, o.event_type, p.usuarios,
       ROUND(100.0 * p.usuarios / LAG(p.usuarios) OVER (ORDER BY o.paso), 1)
         AS conv_vs_paso_anterior,
       ROUND(100.0 * p.usuarios / FIRST_VALUE(p.usuarios) OVER (ORDER BY
o.paso), 1)
         AS conv_vs_inicio
FROM orden o
JOIN pasos p ON p.event_type = o.event_type
ORDER BY o.paso;
```

paso	usuarios	vs paso anterior	vs inicio
1. Ver producto	11	—	100,0 %
2. Carrito	8	72,7 %	72,7 %
3. Checkout	6	75,0 %	54,5 %
4. Compra	4	66,7 %	36,4 %

Explicación: `COUNT(DISTINCT user_id)` cuenta usuarios, no eventos. La tabla VALUES fija el orden de los pasos, porque el orden alfabético no sirve.

Trampa típica: con `COUNT(*)` el primer paso daría 12 en vez de 11, porque el usuario 9 vio un producto dos veces.

Matiz avanzado: esta versión cuenta a cualquiera que llegue a un paso. Un embudo estricto exigiría haber pasado por los anteriores y en orden. Aquí coinciden, pero en datos reales no siempre.

SQL-13 · Retención por cohortes (cohort retention)

```
WITH primera AS (
  SELECT customer_id,
         DATE_TRUNC('month', MIN(order_date))::date AS cohorte
  FROM orders
  WHERE status = 'delivered'
  GROUP BY customer_id
),
actividad AS (
  SELECT DISTINCT customer_id,
         DATE_TRUNC('month', order_date)::date AS mes
  FROM orders
  WHERE status = 'delivered'
),
base AS (
  SELECT p.cohorte, a.customer_id,
         (EXTRACT(YEAR FROM a.mes) - EXTRACT(YEAR FROM p.cohorte)) * 12
         + EXTRACT(MONTH FROM a.mes) - EXTRACT(MONTH FROM p.cohorte) AS mes_n
  FROM primera p
  JOIN actividad a ON a.customer_id = p.customer_id
)
SELECT cohorte,
       COUNT(*) FILTER (WHERE mes_n = 0) AS m0,
       COUNT(*) FILTER (WHERE mes_n = 1) AS m1,
       COUNT(*) FILTER (WHERE mes_n = 2) AS m2,
       COUNT(*) FILTER (WHERE mes_n = 3) AS m3,
       COUNT(*) FILTER (WHERE mes_n = 4) AS m4
FROM base
GROUP BY cohorte
ORDER BY cohorte;
```

cohorte	m0	m1	m2	m3	m4
2026-01	2	1	1	1	1
2026-02	2	1	1	1	0
2026-03	4	2	1	0	0
2026-04	1	0	0	0	0
2026-05	2	0	0	0	0

Cómo se lee: de los 4 clientes que compraron por primera vez en marzo, 2 volvieron en abril (50 %) y 1 en mayo. Los ceros de las cohortes recientes no son malos resultados: todavía no ha pasado ese tiempo.

Explicación: tres pasos. Primero, el mes de la primera compra de cada cliente; segundo, los meses en que cada cliente compró; tercero, la distancia en meses y el recuento por cohorte. `FILTER` cuenta solo las filas que cumplen la condición, y así pivotas sin funciones especiales.

Trampa típica: usar `signup_date` en vez de la primera compra. Eso responde a otra pregunta: la retención desde el registro.

SQL-14 · Nuevos frente a recurrentes

```
WITH primera AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_date)) AS mes_primera
  FROM orders
  WHERE status = 'delivered'
  GROUP BY customer_id
)
SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
       COUNT(DISTINCT o.customer_id) FILTER (
         WHERE DATE_TRUNC('month', o.order_date) = p.mes_primera) AS nuevos,
       COUNT(DISTINCT o.customer_id) FILTER (
         WHERE DATE_TRUNC('month', o.order_date) > p.mes_primera) AS
recurrentes
FROM orders o
JOIN primera p ON p.customer_id = o.customer_id
WHERE o.status = 'delivered'
GROUP BY 1
ORDER BY 1;
```

Resultado (nuevos / recurrentes): enero 2/0, febrero 2/1, marzo 4/2, abril 1/4, mayo 2/3.

Explicación: un cliente es nuevo en el mes de su primera compra y recurrente en cualquier mes posterior.

Para destacar: uno de los dos «nuevos» de mayo es la segunda cuenta de Ana. Si deduplicas, mayo queda en 1 nuevo y 4 recurrentes. Desde abril, la tienda vive sobre todo de la recompra (repeat purchase).

SQL-15 · Clientes reales

```
WITH persona AS (
  SELECT customer_id,
         MIN(customer_id) OVER (PARTITION BY email) AS persona_id
  FROM customers
)
SELECT c.name,
       COUNT(DISTINCT o.order_id)       AS pedidos,
       SUM(oi.quantity * oi.unit_price) AS ingresos
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN persona pe     ON pe.customer_id = o.customer_id
JOIN customers c    ON c.customer_id = pe.persona_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id, c.name
ORDER BY ingresos DESC;
```

Resultado: salen 10 personas en lugar de 11 cuentas. Ana García pasa a tener 4 pedidos y 237,00 €, en vez de aparecer partida en 197,00 y 40,00 €, y sube del sexto al quinto puesto.

Explicación: `MIN(customer_id) OVER (PARTITION BY email)` asigna a todas las cuentas de un mismo email el id más antiguo. Después agrupas por ese id.

En la vida real: antes de unificar, confirma con negocio que el email identifica de verdad a una persona (en las cuentas de empresa, por ejemplo, no siempre).

SQL-16 · Impacto de los descuentos (discounts)

```
SELECT p.product_name,
       COUNT(*) FILTER (WHERE oi.unit_price < p.list_price) AS
lineas_con_descuento,
       SUM((p.list_price - oi.unit_price) * oi.quantity)    AS
euros_descontados,
       ROUND(100.0 * SUM((p.list_price - oi.unit_price) * oi.quantity)
             / SUM(p.list_price * oi.quantity), 1)          AS pct_descuento
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status = 'delivered'
GROUP BY p.product_name
HAVING SUM((p.list_price - oi.unit_price) * oi.quantity) > 0
ORDER BY euros_descontados DESC;
```

producto	líneas con descuento	euros descontados	% descuento
Monitor 27″	2	37,50	2,5 %
Teclado mecánico	1	8,00	2,0 %
Altavoz Bluetooth	1	4,00	1,7 %
Ratón inalámbrico	1	3,00	1,7 %

Explicación: el descuento de cada línea es (precio de catálogo (list price) − precio cobrado) × cantidad. En total son 52,50 €, un 1,85 % de lo que se habría cobrado a precio de catálogo (2.845,00 €).

Lectura: el monitor concentra el 71 % del dinero descontado. Si hay que revisar la política de descuentos, se empieza por ahí.

SQL-17 · Media frente a mediana (median)

```
WITH totales AS (
  SELECT o.order_id, SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.order_id
)
SELECT ROUND(AVG(total), 2) AS media,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total) AS mediana,
       MIN(total) AS minimo,
       MAX(total) AS maximo
FROM totales;
```

Resultado: media 126,93 €, mediana 97,50 €, mínimo 27,00 € y máximo 330,00 €, sobre 22 pedidos.

Por qué difieren: unos pocos pedidos con monitor, de 225 € o más, tiran de la media hacia arriba. La mediana representa mejor el pedido típico; la media sirve para calcular ingresos, porque media × pedidos = total.

En un dashboard: pon las dos, o la mediana con una nota. Saber cuándo usar cada una es una pregunta muy frecuente.

Bloque 2 · Excel y Google Sheets

Se supone que cada tabla está en su pestaña con los encabezados en la fila 1 y las columnas en el orden del script: en orders, A = order_id, B = customer_id, C = order_date y D = status; en order_items, A = order_id, B = product_id, C = quantity y D = unit_price. Las fórmulas van en español, que usa punto y coma, con su versión en inglés debajo.

XLS-1 · Importe (amount) y estado de cada línea

En order_items, añade tres columnas y copia las fórmulas hacia abajo:

```
E2 (importe):  =C2*D2
F2 (fecha):    =BUSCARX(A2; orders!A:A; orders!C:C)     |  =XLOOKUP(A2,
orders!A:A, orders!C:C)
G2 (estado):   =BUSCARX(A2; orders!A:A; orders!D:D)     |  =XLOOKUP(A2,
orders!A:A, orders!D:D)
Ingresos:      =SUMAR.SI.CONJUNTO(E:E; G:G; "delivered") |  =SUMIFS(E:E, G:G,
"delivered")
```

Resultado: 2.792,50 €, igual que SQL-1.

Explicación: BUSCARX busca el order_id de la línea en la columna A de orders y devuelve el valor de la misma fila en la columna que indiques. Es el equivalente a un JOIN. SUMAR.SI.CONJUNTO suma la columna E solo donde la columna G vale «delivered».

Trampa típica: usar BUSCARV con un número de columna fijo. Si alguien inserta una columna en medio, la fórmula devuelve datos equivocados sin avisar; BUSCARX no tiene ese problema.

XLS-2 · Ingresos (revenue) por mes

```
H2 (mes):  =FIN.MES(F2; -1) + 1     |  =EOMONTH(F2, -1) + 1
```

Formatea la columna H como fecha «mmm-aaaa». Después escribe los meses en J2:J6 (01/01/2026 a 01/05/2026) y en K2:

```
=SUMAR.SI.CONJUNTO(E:E; G:G; "delivered"; H:H; J2)   |  =SUMIFS(E:E, G:G,
"delivered", H:H, J2)
```

Resultado: 420,00; 217,00; 890,00; 493,00 y 772,50 €, como en SQL-8.

Explicación: FIN.MES(fecha; −1) da el último día del mes anterior; sumándole 1 obtienes el día 1 del mes de la fecha. Es el equivalente de DATE_TRUNC. SUMAR.SI.CONJUNTO admite varias condiciones a la vez.

XLS-3 · Tabla dinámica (pivot table) por categoría

```
I2 (categoría):  =BUSCARX(B2; products!A:A; products!C:C)   |  =XLOOKUP(B2, products!A:A, products!C:C)
```

Crea la tabla dinámica sobre order_items con: categoría en filas, suma de importe en valores y estado = delivered como filtro. Añade el importe una segunda vez a valores y míralo como «% del total general» (en Excel, Mostrar valores como; en Sheets, Mostrar como).

categoría	ingresos (€)	% del total
Monitores	1.462,50	52,4 %
Periféricos	794,00	28,4 %
Audio	536,00	19,2 %

Para destacar: un solo producto, el monitor, aporta más de la mitad de los ingresos. Es un dato que un director quiere conocer.

XLS-4 · Limpieza (data cleaning) de clientes

En customers (D = city, C = email):

```
G2 (ciudad limpia):  =SI(ESPACIOS(D2)=""; "Desconocida"; NOMPROPIO(ESPACIOS(D2)))
                     =IF(TRIM(D2)="", "Desconocida", PROPER(TRIM(D2)))
H2 (duplicado):      =SI(CONTAR.SI(C:C; C2)>1; "Duplicado"; "")
                     =IF(COUNTIF(C:C, C2)>1, "Duplicado", "")
```

Explicación: ESPACIOS quita los espacios sobrantes y NOMPROPIO pone mayúscula inicial. Se comprueba primero si la celda queda vacía para poner «Desconocida». CONTAR.SI cuenta cuántas veces aparece el email en toda la columna.

Trampa típica: si al exportar desde DBeaver el nulo aparece como el texto «NULL», la celda no está vacía y la fórmula escribiría «Null». Revisa siempre cómo llegan los nulos a la hoja.

Bloque 3 · Power BI y DAX

Las fórmulas usan comas. Si tu Power BI está configurado con separadores regionales, cámbialas por punto y coma.

BI-1 · El modelo

- **Tablas de hechos (fact tables)** (registran lo que pasa y crecen cada día): order_items, con cantidades e importes, y orders, con la fecha y el estado de cada pedido.
- **Tablas de dimensiones (dimension tables)** (describen el contexto): customers, products y Calendario.
- **Relaciones, todas de uno a varios y en una sola dirección:** products[product_id] → order_items[product_id]; orders[order_id] → order_items[order_id]; customers[customer_id] → orders[customer_id]; Calendario[Date] → orders[order_date].

Por qué un calendario: las funciones de tiempo como DATEADD o TOTALYTD necesitan una tabla de fechas (date table) continua y marcada como tabla de fechas. Además, permite mostrar meses sin ventas. Se crea en Modelado → Nueva tabla:

```
Calendario =
ADDCOLUMNS(
    CALENDAR(DATE(2026, 1, 1), DATE(2026, 12, 31)),
    "Año", YEAR([Date]),
    "Mes", FORMAT([Date], "yyyy-mm")
)
```

Después, clic derecho en la tabla → Marcar como tabla de fechas.

Para destacar: este es el modelo en estrella (star schema): los filtros bajan desde las dimensiones hacia los hechos. Explicarlo bien demuestra que entiendes por qué los números cuadran.

BI-2 · Medida de ingresos (revenue)

```
Ingresos =
CALCULATE(
    SUMX(order_items, order_items[quantity] * order_items[unit_price]),
    orders[status] = "delivered"
)
```

Resultado: 2.792,50.

Explicación: SUMX recorre order_items fila a fila, multiplica cantidad por precio y suma el resultado. CALCULATE añade el filtro de estado sobre orders, que llega a order_items a través de la relación.

Trampa típica: `SUM(order_items[unit_price])` ignora la cantidad: en las líneas con 2 unidades sumaría solo una.

BI-3 · Ticket medio (average order value, AOV)

```
Ticket medio =
DIVIDE(
    [Ingresos],
    CALCULATE(DISTINCTCOUNT(orders[order_id]), orders[status] = "delivered")
)
```

Resultado: 126,93.

Explicación: reutilizas la medida [Ingresos] y la divides entre el número de pedidos entregados. DIVIDE devuelve un valor vacío si el denominador es cero, en lugar de un error.

BI-4 · Peso de cada categoría

```
% sobre total =
DIVIDE(
    [Ingresos],
    CALCULATE([Ingresos], REMOVEFILTERS(products[category]))
)
```

Resultado: Monitores 52,4 %, Periféricos 28,4 % y Audio 19,2 %.

Explicación: en cada fila de la tabla el visual filtra una categoría. En el denominador, REMOVEFILTERS quita ese filtro, así que calcula el total de todas las categorías. ALL(products[category]) también vale.

BI-5 · Crecimiento mensual (month-over-month growth)

```
Crecimiento mensual % =
VAR Actual   = [Ingresos]
VAR Anterior = CALCULATE([Ingresos], DATEADD(Calendario[Date], -1, MONTH))
RETURN
    DIVIDE(Actual - Anterior, Anterior)
```

Ponle formato de porcentaje y úsala en una tabla con Calendario[Mes].

Resultado: enero vacío, febrero −48,3 %, marzo +310,1 %, abril −44,6 % y mayo +56,7 %, como en SQL-8.

Explicación: VAR guarda resultados intermedios y hace la medida más legible. DATEADD desplaza un mes hacia atrás las fechas del contexto, así que [Ingresos] se calcula para el mes anterior.

Trampa típica: si el calendario no está marcado como tabla de fechas o no está relacionado con orders, DATEADD devuelve vacío o resultados raros.

BI-6 · Pregunta teórica

Respuesta modelo: «Una columna calculada (calculated column) se calcula fila a fila al actualizar el modelo y se guarda, así que ocupa memoria; la uso para agrupar o filtrar, por ejemplo un tramo de precio. Una medida se calcula en el momento según los filtros del visual y no se guarda; la uso para cifras agregadas, como ingresos o ticket medio. Mi regla: si va en un eje o en un filtro, columna; si es un número que se suma o se divide, medida. CALCULATE evalúa una expresión cambiando el contexto de filtro (filter context): añade, quita o sustituye filtros. Es la función más importante de DAX.»

Bloque 4 · Python y pandas

PY-1 · Cargar y auditar

```
import pandas as pd
from sqlalchemy import create_engine

# Cambia usuario y contraseña por los de tu PostgreSQL
engine =
create_engine("postgresql+psycopg2://usuario:contraseña@localhost:5432/tienda
")

customers = pd.read_sql("SELECT * FROM customers", engine, parse_dates=
["signup_date"])
orders    = pd.read_sql("SELECT * FROM orders", engine, parse_dates=
["order_date"])
items     = pd.read_sql("SELECT * FROM order_items", engine)
products  = pd.read_sql("SELECT * FROM products", engine)

items["unit_price"] = items["unit_price"].astype(float)

for nombre, df in {"customers": customers, "orders": orders,
                   "items": items, "products": products}.items():
    print(nombre, df.shape)
    print(df.dtypes, "\n")
    print(df.isna().sum(), "\n")
```

Resultado: customers (12, 6), orders (25, 4), items (32, 4) y products (6, 4). El único nulo está en customers.city.

Explicación: `parse_dates` convierte las fechas a tipo fecha; sin ello pueden llegar como texto y fallar al agrupar por mes. Según la versión del driver, los NUMERIC de PostgreSQL llegan como objetos Decimal, que dan problemas al mezclarlos con decimales normales; por eso se convierte el precio a float.

Para destacar: empezar siempre por forma, tipos y nulos demuestra método. Es lo primero que mira un entrevistador en un caso para casa (take-home assignment).

PY-2 · Limpiar clientes

```
clientes = customers.copy()
clientes["city"] =
clientes["city"].str.strip().str.title().fillna("Desconocida")

print(clientes[clientes.duplicated("email", keep=False)][["customer_id",
"email"]])

personas = (clientes.sort_values("customer_id")
                    .drop_duplicates("email", keep="first"))
print(len(clientes), "cuentas ->", len(personas), "personas")
```

Resultado: las cuentas 1 y 10 comparten ana@mail.com; quedan 11 personas de 12 cuentas.

Explicación: `.str` aplica métodos de texto a toda la columna (`title()` equivale a INITCAP). `duplicated(keep=False)` marca todas las copias, que es lo que quieres ver; `drop_duplicates(keep="first")` se queda con la cuenta más antigua porque antes ordenas por id.

Trampa típica: trabajar sobre `customers` sin `.copy()` y modificar sin querer los datos originales.

PY-3 · Ingresos (revenue) mensuales

```
items["importe"] = items["quantity"] * items["unit_price"]
ventas = items.merge(orders, on="order_id", how="inner")
entregadas = ventas[ventas["status"] == "delivered"]

mensual = (entregadas
           .groupby(entregadas["order_date"].dt.to_period("M"))["importe"]
           .sum()
           .to_frame("ingresos"))
mensual["crecimiento_pct"] = (mensual["ingresos"].pct_change() *
100).round(1)
print(mensual)
```

Resultado: 420,0 / 217,0 / 890,0 / 493,0 / 772,5, con crecimientos de −48,3; +310,1; −44,6 y +56,7 %.

Explicación: `merge` es el JOIN de pandas. `dt.to_period("M")` equivale a DATE_TRUNC por mes y `pct_change()`, a la combinación de LAG y división.

Trampa típica: después del merge hay una fila por línea, no por pedido. Si cuentas pedidos con `len()` cuentas líneas; usa `nunique()` sobre order_id.

PY-4 · Matriz de cohortes

```
pedidos_ok = orders[orders["status"] == "delivered"].copy()
pedidos_ok["mes"] = pedidos_ok["order_date"].dt.to_period("M")
pedidos_ok["cohorte"] = pedidos_ok.groupby("customer_id")
["mes"].transform("min")
pedidos_ok["mes_n"] = (pedidos_ok["mes"] -
pedidos_ok["cohorte"]).apply(lambda d: d.n)

tabla = pedidos_ok.pivot_table(index="cohorte", columns="mes_n",
                               values="customer_id", aggfunc="nunique",
fill_value=0)
retencion = (tabla.div(tabla[0], axis=0) * 100).round(0)
print(retencion)
```

cohorte	m0	m1	m2	m3	m4
2026-01	100	50	50	50	50
2026-02	100	50	50	50	0
2026-03	100	50	25	0	0
2026-04	100	0	0	0	0
2026-05	100	0	0	0	0

Explicación: `transform("min")` da a cada fila el primer mes de su cliente sin perder filas; es el equivalente de `MIN() OVER (PARTITION BY ...)`. Restar dos periodos devuelve un desplazamiento y `.n` lo convierte en número de meses. `div(tabla[0], axis=0)` divide cada fila entre su mes 0.

PY-5 · Función de KPIs

```
def resumen_kpis(orders, items, desde, hasta):
    """KPIs de pedidos entregados entre dos fechas, ambas incluidas."""
    o = orders[(orders["status"] == "delivered")
               & (orders["order_date"].between(desde, hasta))]
    i = items[items["order_id"].isin(o["order_id"])]
    ingresos = float((i["quantity"] * i["unit_price"]).sum())
    pedidos = o["order_id"].nunique()
    return {
        "ingresos": round(ingresos, 2),
        "pedidos": pedidos,
        "ticket_medio": round(ingresos / pedidos, 2) if pedidos else 0.0,
        "clientes": o["customer_id"].nunique(),
    }

print(resumen_kpis(orders, items, "2026-03-01", "2026-03-31"))
print(resumen_kpis(orders, items, "2027-01-01", "2027-01-31"))
```

Resultado: marzo da 890,0 € de ingresos, 6 pedidos, 148,33 € de ticket medio (average order value, AOV) y 6 clientes. El mes sin datos devuelve ceros sin romperse.

Explicación: la función tiene una descripción de una línea, no modifica los datos de entrada y protege la división entre cero. Probarla con un caso vacío es justo lo que valoran en una entrevista: demuestra que piensas en los casos límite.

PY-6 · Dos gráficos

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

mensual["ingresos"].plot(ax=axes[0], marker="o")
axes[0].set_title("Ingresos mensuales (pedidos entregados)")
axes[0].set_ylabel("€")
axes[0].set_xlabel("")

por_categoria = (entregadas.merge(products, on="product_id")
                 .groupby("category")["importe"].sum().sort_values())
por_categoria.plot.barh(ax=axes[1])
axes[1].set_title("Ingresos por categoría")
axes[1].set_xlabel("€")
axes[1].set_ylabel("")

plt.tight_layout()
plt.show()
```

Por qué cada tipo: la línea muestra cómo evoluciona algo en el tiempo, porque une puntos consecutivos. Las barras horizontales ordenadas comparan categorías y dejan leer bien los nombres largos. Cada gráfico lleva un título que dice qué muestra y las unidades en el eje.

Trampa típica: usar un gráfico de tarta para comparar categorías. El ojo compara mal ángulos y áreas; las barras se leen mucho mejor.

Bloque 5 · Estadística y tests A/B

EST-1 · El sueldo «medio»

Resultado: media 44.142,86 € y mediana (median) 32.000 €.

Explicación: un solo sueldo de 120.000 € arrastra la media (mean) hacia arriba: seis de las siete personas cobran menos que la media. Para describir lo que cobra alguien normal del equipo, la mediana. Regla general: con datos asimétricos o con valores extremos (sueldos, precios de vivienda, importes de pedido), usa la mediana; la media sirve cuando necesitas totales.

EST-2 · Qué significa un p-valor (p-value)

Respuesta modelo: «Si en realidad no hubiera ninguna diferencia entre A y B, la probabilidad de ver por azar una diferencia tan grande como esta, o mayor, sería del 3 %. Como es menor que el umbral que fijamos, el 5 %, consideramos la diferencia estadísticamente significativa (statistically significant).»

Qué falla en cada frase:

1. **«Hay un 97 % de probabilidad de que B sea mejor.»** Falso. El p-valor se calcula suponiendo que no hay diferencia; no es la probabilidad de que una variante sea mejor.
2. **«El efecto es grande.»** Falso. El p-valor no mide el tamaño del efecto (effect size). Con muestras enormes, diferencias minúsculas dan p-valores muy pequeños. El tamaño se mira en la diferencia y en su intervalo de confianza (confidence interval).
3. **«Con p = 0,06 habría demostrado que no hay diferencia.»** Falso. No ser significativo no prueba que no haya efecto: puede que la muestra fuera pequeña. Ausencia de evidencia no es evidencia de ausencia.

EST-3 · ¿Ha ganado la variante B?

```
z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}\,(1-\hat{p})\left(\frac{1}
{n_A}+\frac{1}{n_B}\right)}}
```

Cálculo: A convierte un 10,0 % y B un 10,8 %. La proporción conjunta (pooled proportion) es 2.080 / 20.000 = 0,104. El error estándar (standard error) es la raíz de 0,104 × 0,896 × (1/10.000 + 1/10.000), es decir, 0,00432. Entonces z = 0,008 / 0,00432 = 1,85 y el p-valor bilateral (two-sided) es 0,064.

```
from statsmodels.stats.proportion import proportions_ztest
z, p = proportions_ztest([1080, 1000], [10000, 10000])
print(round(z, 2), round(p, 3))   # 1.85 0.064
```

Conclusión: p = 0,064 es mayor que 0,05, así que la diferencia no es significativa. B parece mejor, con 0,8 puntos más, pero con esta muestra una diferencia así sale por azar más de un 6 % de las veces. No se puede declarar ganadora. EST-4 explica por qué faltaban datos.

EST-4 · ¿Cuántos usuarios necesito?

```
n = \frac{\left(z_{1-\alpha/2} + z_{1-\beta}\right)^2 \left[p_1(1-p_1) +
p_2(1-p_2)\right]}{(p_2 - p_1)^2}
```

Cálculo: $(1{,}96 + 0{,}84)^2 \times (0{,}10 \times 0{,}90 + 0{,}11 \times 0{,}89) / 0{,}01^2$, que da unos **14.750 usuarios por grupo**. Con statsmodels (`NormalIndPower` y `proportion_effectsize`) salen 14.744, prácticamente lo mismo.

Lectura: el test de EST-3 tenía 10.000 usuarios por grupo, menos de lo necesario para detectar con fiabilidad una subida de un punto. Por eso no fue concluyente. La regla es calcular el tamaño de muestra (sample size) antes de empezar y no parar el test antes de tiempo.

EST-5 · Intervalo de confianza

Cálculo: el error estándar es la raíz de 0,108 × 0,892 / 10.000, es decir, 0,0031. El intervalo es 10,8 % ± 1,96 × 0,31 %: **entre 10,19 % y 11,41 %.**

Interpretación: la conversión real de B está, de forma plausible, entre el 10,2 % y el 11,4 %. Con más rigor: si repitiéramos el experimento muchas veces, el 95 % de los intervalos calculados así contendrían el valor real.

Para destacar: el intervalo de la diferencia B − A va de −0,05 a +1,65 puntos. Incluye el cero, y por eso EST-3 no salía significativo. Los intervalos cuentan más que un p-valor: dan el tamaño posible del efecto.

EST-6 · La variante que gana y pierde a la vez

Cálculo: por segmentos, B gana en móvil (11,5 % frente a 10,0 %) y en escritorio (27,0 % frente a 25,0 %). En el total, A convierte un 22,0 % (550 / 2.500) y B un 14,6 % (365 / 2.500).

Por qué pasa: es la paradoja de Simpson (Simpson's paradox). B recibió sobre todo tráfico móvil (2.000 de 2.500 usuarios), que convierte mucho menos; A recibió sobre todo escritorio. El total mezcla el efecto de la variante con la composición del tráfico.

Qué le dirías al equipo: el reparto entre variantes no fue aleatorio, así que el test no es válido tal cual. Hay que revisar cómo se asignaban los usuarios y repetirlo con un reparto equilibrado. Mientras tanto, la comparación justa es dentro de cada segmento, donde B gana.

EST-7 · Correlación y causalidad (correlation vs. causation)

Respuesta modelo: «Que los usuarios de la app gasten más no significa que la app les haga gastar más. Lo más probable es lo contrario: los clientes que ya compran mucho son los que se molestan en instalarla. Si obligamos a todos, podemos perder a quien prefiere la web. Propongo un experimento: ofrecer un incentivo para instalar la app a un grupo elegido al azar y comparar su gasto con un grupo de control durante unas semanas.»

Qué se valora: nombrar la causalidad inversa (reverse causality) o las variables de confusión (confounding variables) y proponer un experimento en vez de quedarte en «correlación no implica causalidad».

EST-8 · Mirar el test cada día (peeking)

Problema: cada vez que miras el test y puedes pararlo, tienes otra oportunidad de encontrar un falso positivo (false positive). En una simulación con dos variantes idénticas y 20 revisiones, el test sale «ganador» por azar en torno al 26 % de las veces, en lugar del 5 % prometido.

Solución: fija el tamaño de muestra antes de empezar (EST-4) y analiza solo al final. Si de verdad necesitas mirar sobre la marcha, usa métodos secuenciales (sequential testing), que están diseñados para eso.

Bloque 6 · Casos de negocio

NEG-1 · Los ingresos (revenue) han caído un 10 %

Estructura de la respuesta:

1. **Comprobar el dato.** ¿Ha cambiado el seguimiento (tracking), falta cargar algún día, ha cambiado la definición de venta? ¿Tienen los dos meses los mismos días y fines de semana?
2. **Descomponer.** Ingresos = pedidos × ticket medio (average order value, AOV), y pedidos = visitas × conversión. Hay que ver qué pieza ha caído.
3. **Segmentar la pieza que cae:** por canal, dispositivo, región, categoría y clientes nuevos o recurrentes.
4. **Buscar causas internas:** cambios de precio, roturas de stock, caídas de la web, campañas que terminaron.
5. **Buscar causas externas:** estacionalidad (compara con el mismo mes del año anterior), competencia, festivos.
6. **Cerrar con hipótesis ordenadas** por probabilidad y el siguiente análisis que harías.

Ejemplo con TiendaDemo: de marzo a abril los ingresos caen un 44,6 %. Los pedidos solo bajan de 6 a 5 (−17 %), pero el ticket medio pasa de 148,33 a 98,60 € (−34 %). La causa principal es el mix de producto: en marzo se vendieron dos monitores y en abril uno. Ese es el tipo de conclusión que buscan.

NEG-2 · Medir una función nueva

- **Adopción:** porcentaje de usuarios activos que añaden al menos un producto a la lista en sus primeras semanas.
- **Uso:** productos guardados por usuario y porcentaje de usuarios que vuelven a consultar su lista.
- **Impacto (la métrica principal):** porcentaje de productos guardados que acaban comprándose e ingresos por usuario, comparando con y sin la función.
- **Guardarraíles (lo que no debe empeorar):** conversión global, días hasta la compra (la lista no debería retrasar compras) y velocidad de la página.

Para destacar: para saber si la lista causa más ventas, hay que lanzarla con un test A/B. Comparar a quien la usa con quien no, sin más, cae en el error de EST-7.

NEG-3 · ¿Sale rentable captar clientes?

Cálculo: LTV = 60 € × 3 pedidos al año × 30 % de margen × 2 años = **108 €**. LTV/CAC = 108 / 25 = **4,3**. Cada cliente genera 54 € de margen al año, así que recuperas los 25 € de captación en unos 6 meses.

Lectura: una referencia habitual es que la ratio supere 3, así que es un buen negocio, siempre que las hipótesis se cumplan.

Trampa típica: calcular el LTV con los ingresos (360 €) en vez del margen. Saldría una ratio de 14,4, que sobreestima muchísimo. También vale comentar que dos años de vida media es una hipótesis que hay que comprobar con datos de retención (retention).

NEG-4 · La métrica principal

Respuesta modelo: «Número de ventas completadas entre particulares por semana. En un marketplace, el valor se crea cuando comprador y vendedor cierran una operación; esa métrica recoge las dos caras a la vez. Usuarios registrados es una métrica de vanidad (vanity metric): solo crece, incluye a gente que nunca vuelve y no dice si el producto funciona.»

Métricas de apoyo: anuncios publicados (oferta), contactos entre usuarios (demanda), porcentaje de anuncios que se venden y días hasta vender.

NEG-5 · Dashboard para dirección

Los cinco indicadores: ingresos, pedidos, ticket medio, porcentaje de ingresos de clientes recurrentes (returning customers) y tasa de cancelación y devolución. Cada uno con su variación frente al mes anterior y frente al objetivo.

Organización: arriba, cinco tarjetas con los indicadores; en medio, la evolución mensual de ingresos; abajo, el desglose por canal y por categoría. Filtros de fechas y canal.

Por qué así: en cinco segundos el director debe saber si van bien; si quiere saber por qué, baja al desglose. Pocos indicadores bien elegidos valen más que veinte gráficos.

NEG-6 · Dar una mala noticia

Mensaje modelo:

> Hola, Marta: ya tenemos el resultado del test del botón nuevo. B convirtió un 10,8 % frente al 10,0 % de A, pero la diferencia todavía no es concluyente: con 10.000 usuarios por grupo, algo así puede salir por azar. Para confirmarlo necesitamos unos 15.000 usuarios por grupo, unas semanas más con el tráfico actual. Propongo seguir hasta ese tamaño sin mirar resultados intermedios. Si el cambio no tiene coste, también podemos lanzarlo ya, sabiendo que la mejora, si existe, es pequeña.

Qué se valora: el resultado va en la primera frase, sin jerga (ni «p-valor (p-value)» ni «z»), y el mensaje termina con opciones y una recomendación.

NEG-7 · Estimación rápida (Fermi estimate)

Razonamiento en voz alta (cada cifra es una hipótesis):

1. Barcelona tiene unos 1,6 millones de habitantes y unos 650.000 hogares, a 2,5 personas por hogar.
2. Supongamos que el 40 % de los hogares pide comida a domicilio alguna vez al mes: 260.000 hogares.
3. Esos hogares piden de media 3 veces al mes: 780.000 pedidos al mes, unos 26.000 al día en toda la ciudad.
4. Si la app líder tiene entre el 40 y el 50 % del mercado, gestiona **entre 10.000 y 13.000 pedidos al día**.
5. Comprobación: falta sumar a los turistas y oficinas, y los fines de semana habrá picos, así que la cifra real será algo mayor.

Qué se valora: no la cifra exacta, sino la estructura, que digas las hipótesis en voz alta y que compruebes si el resultado es razonable.

Bloque 7 · IA aplicada

IA-1 · Encuentra los errores

Los cuatro errores:

1. Agrupa por día (`order_date`) en vez de por mes.
2. Usa `AVG` en vez de `SUM`: calcula la media (mean) por línea, no los ingresos (revenue).
3. `COUNT(*)` cuenta líneas de pedido, no pedidos: un pedido con dos productos cuenta doble.
4. No filtra el estado: incluye pedidos cancelados y devueltos.

```
SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
       SUM(oi.quantity * oi.unit_price)        AS ingresos,
       COUNT(DISTINCT o.order_id)              AS pedidos
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'delivered'
GROUP BY 1
ORDER BY 1;
```

Resultado corregido: los mismos ingresos que SQL-8 y los mismos pedidos que SQL-2. La versión de la IA daba, por ejemplo, 55 € de «ingresos» y 2 «pedidos» el 8 de enero, cuando era un solo pedido de 110 €.

Lección: la consulta (query) de la IA se ejecuta sin errores, y eso es lo peligroso. Comprueba siempre contra un total conocido: aquí, que la suma de los meses dé 2.792,50 €.

IA-2 · Escribe el prompt

Prompt modelo:

> Trabajo en PostgreSQL 16. Tengo la tabla orders(order_id INT, customer_id INT, order_date DATE, status TEXT), donde status puede ser 'delivered', 'cancelled' o 'returned'. Quiero una tabla de retención por cohortes (cohort retention). La cohorte de cada cliente es el mes de su primera compra entregada. Las columnas m0 a m4 son el número de clientes de esa cohorte que hicieron una compra entregada 0, 1, 2, 3 y 4 meses después. Solo cuentan los pedidos con status = 'delivered' y cada cliente cuenta una vez por mes aunque haga varios pedidos. Usa CTEs con nombres claros, explica cada paso y dime qué comprobaciones harías para validar el resultado.

Por qué funciona: el esquema evita que invente columnas; las definiciones exactas (qué es cohorte, qué cuenta como compra) eliminan la ambigüedad; el dialecto evita funciones de otro motor; y pedir explicación y comprobaciones te da material para revisar.

IA-3 · ¿Es buena tu clasificación (classification)?

Cálculo:

- **Exactitud (accuracy):** 41 / 50 = 82 %.
- **Precisión (precision) de «retraso»:** 14 / 16 = 87,5 %. De lo que el modelo marca como retraso, acierta casi 9 de cada 10.
- **Exhaustividad de «retraso»:** 14 / 18 = 77,8 %. De los retrasos reales, detecta 8 de cada 10.

¿Es suficiente? Para ver tendencias, sí, con una advertencia: el modelo se deja uno de cada cinco retrasos, así que infravalora cuántas quejas son por retraso. Además, con solo 50 reseñas revisadas, el 82 % tiene un margen de unos ±11 puntos. Si la conclusión depende de contar retrasos, mejora las instrucciones con ejemplos de retrasos mal clasificados y revisa una muestra mayor.

Para destacar: la exactitud sola engaña cuando una categoría es mucho más frecuente que otras. Por eso se mira precisión y exhaustividad por categoría.

IA-4 · Datos de clientes en un chat de IA

Respuesta modelo: «No sin permiso. Nombres y emails son datos personales (personal data, PII) protegidos por el RGPD (GDPR), y una herramienta externa puede guardarlos. Primero miraría la política de la empresa y qué herramientas están aprobadas. Para pedir ayuda con el SQL, basta con pasar el esquema de las tablas y unas filas inventadas. Si necesito que la IA trabaje con los datos reales, los anonimizo o uso una herramienta aprobada por la empresa o un modelo local.»

Qué se valora: que conozcas el riesgo y propongas alternativas prácticas en lugar de un simple «no».

Bloque 8 · Git

GIT-1 · Has subido una clave

El orden importa:

1. **Revoca la clave ya** en el servicio que la emitió y genera una nueva. Es lo primero porque hay programas que rastrean GitHub en busca de claves; aunque la borres, ya pudo copiarse y sigue en el historial.
2. **Saca el archivo del repositorio (repository)** y que Git lo ignore a partir de ahora:

```
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Remove .env and ignore it"
git push
```

3. **Si el repositorio es público**, limpia también el historial con una herramienta como git filter-repo. Como la clave ya está revocada, esto es higiene, no la solución.
4. **Para el futuro:** guarda las claves en variables de entorno (environment variables) y sube solo un `.env.example` sin valores.

Qué se valora: que el primer paso sea revocar la clave. Quien empieza por borrar el archivo no ha entendido el riesgo.

GIT-2 · Deshacer el último commit

Si todavía no has hecho push:

```
git reset --soft HEAD~1
git restore --staged archivo_que_sobraba
git commit -m "Mensaje correcto"
```

`reset --soft` deshace el commit pero conserva los cambios preparados; después quitas el archivo que sobraba y vuelves a hacer el commit.

Si ya hiciste push:

```
git revert HEAD
```

`revert` crea un commit nuevo que deshace el anterior sin reescribir el historial que otras personas pueden haber descargado.

Trampa típica: hacer `reset` y luego `push --force` en una rama compartida. Rompe el trabajo de tus compañeros.

GIT-3 · Conflicto al fusionar (merge conflict)

```
<<<<<<< HEAD
-- tu versión (la de la rama en la que estás)
=======
-- la versión de la rama que estás fusionando
>>>>>>> nombre-de-la-rama
```

Cómo resolverlo: abre el archivo, decide qué se queda (una versión, la otra o una mezcla), borra las tres marcas, comprueba que la consulta (query) sigue funcionando y termina con:

```
git add consultas.sql
git commit
```

VS Code muestra botones para aceptar una versión, la otra o ambas, lo que facilita mucho el trabajo.

Qué se valora: decir que pruebas el resultado antes de cerrar el merge. Un conflicto mal resuelto puede dar un SQL que se ejecuta pero calcula mal.

Bloque 9 · Inglés para entrevistas

Las respuestas modelo son plantillas: cambia todo lo que no sea verdad para ti y lo que va entre corchetes. Apréndete la estructura, no el texto.

EN-1 · «Tell me about yourself»

Estructura: presente (quién eres) → pasado (lo que has hecho que sirve para el puesto) → futuro (por qué este puesto).

> I'm a software developer with a degree in video game development. For the last few years I've been building automation tools for small businesses, such as appointment reminders sent by WhatsApp. Working with those clients, I became more and more interested in the data behind the tools: which reminders reduced no-shows, which customers came back, and how to turn that into decisions. So I decided to focus on data analysis. Over the last three months I've built three portfolio projects with SQL, Python, Power BI and dbt, including a cohort analysis and an A/B test. I'm looking for a role where I can combine my technical background with business questions, and [company] caught my attention because [reason].

Por qué funciona: convierte tu experiencia de desarrollador en una ventaja, explica el cambio con un motivo concreto y termina mirando al puesto, no a tu pasado. Dura algo más de un minuto.

EN-2 · «Walk me through a project»

Estructura: pregunta de negocio → datos → qué hiciste → resultado con una cifra → recomendación → qué harías después.

> The project I'm proudest of is an analysis of a Brazilian e-commerce dataset with around 100,000 orders. The business question was: what makes customers buy again, and how do late deliveries affect reviews? I loaded the data into PostgreSQL, built a cohort retention analysis in SQL and a Power BI dashboard. I also used a language model to classify review comments by reason, and I checked its accuracy by hand on a sample of 50 reviews. The main finding was that [your result, for example: late orders received low ratings X times more often]. My recommendation was [your recommendation]. If I had more time, I would [next step].

Por qué funciona: empieza por la pregunta de negocio, no por las herramientas; da una cifra concreta y menciona cómo validaste el trabajo de la IA, que es justo lo que hoy quieren oír.

EN-3 · «Explain this to a non-technical stakeholder»

> We tested a new button against the current one. The new button did slightly better, 10.8% versus 10%, but the difference is small enough that it could just be luck. It's like flipping a coin ten times and getting six heads: that doesn't prove the coin is unfair. To be confident, we need more users, about 15,000 per group instead of 10,000. So my recommendation is to keep the test running until we reach that number, and then decide.

Por qué funciona: cero jerga, una comparación cotidiana y una recomendación al final.

Frases útiles para cualquier respuesta: *The key takeaway is… · What the data shows is… · One caveat is… · To be confident, we'd need… · I'd recommend… · If I had more time, I would…*

Bloque 10 · Análisis on-chain con BigQuery

Estas tres consultas (queries) están revisadas, pero no se han podido ejecutar aquí contra BigQuery. Antes de lanzarlas, mira la estimación de bytes del editor: con el filtro de fechas deberían procesar pocos gigas.

OC-1 · Direcciones activas (active addresses)

```
SELECT DATE(block_timestamp) AS dia,
       COUNT(DISTINCT from_address) AS direcciones_activas
FROM `bigquery-public-data.crypto_ethereum.transactions`
WHERE block_timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY dia
ORDER BY dia;
```

Explicación: la tabla está particionada (partitioned) por día, así que el filtro sobre `block_timestamp` hace que BigQuery solo lea la última semana en lugar de todo el historial. Pedir solo las columnas necesarias reduce todavía más los bytes, porque los datos se guardan por columnas.

Matices: la definición importa. Aquí «activa» es haber enviado una transacción; si cuentas también las que reciben, el número sube. El primer y el último día salen incompletos: en un informe (report), filtra días enteros.

OC-2 · Volumen de USDT

```
SELECT DATE(block_timestamp) AS dia,
       SUM(CAST(value AS BIGNUMERIC)) / 1e6 AS usdt_transferidos,
       COUNT(*) AS transferencias
FROM `bigquery-public-data.crypto_ethereum.token_transfers`
WHERE token_address = '0xdac17f958d2ee523a2206206994597c13d831ec7'
  AND block_timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY dia
ORDER BY dia;
```

Explicación: la columna `value` se guarda como texto porque los importes en unidades mínimas no caben en un entero normal; por eso se convierte a BIGNUMERIC. Se divide entre un millón porque USDT tiene 6 decimales: en la blockchain, 1 USDT se guarda como 1.000.000. Las direcciones van en minúsculas.

Trampa típica: dividir entre 10^18, que son los decimales de ETH, y obtener cifras un billón de veces más pequeñas. Y para destacar en la entrevista: volumen transferido no es actividad económica real, porque incluye movimientos internos de exchanges y de bots.

OC-3 · Comisiones

```
SELECT DATE(block_timestamp) AS dia,
       AVG(CAST(receipt_gas_used AS BIGNUMERIC) *
receipt_effective_gas_price) / 1e18
         AS comision_media_eth
FROM `bigquery-public-data.crypto_ethereum.transactions`
WHERE block_timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY dia
ORDER BY dia;
```

Explicación: la comisión de una transacción es el gas usado por el precio efectivo del gas, en wei; un ETH son 10^18 wei. Se usa el precio efectivo porque es lo que se pagó de verdad. El CAST evita que la multiplicación de dos enteros grandes desborde.

Para destacar: unas pocas transacciones carísimas inflan la media (mean), igual que en EST-1. La mediana (median) diaria se calcula en BigQuery con `APPROX_QUANTILES(comision, 2)[OFFSET(1)]` y describe mejor lo que paga un usuario normal.

Bloque 11 · Solución del simulacro

Pregunta 1 · Evolución

Los ingresos (revenue) suman 2.792,50 € de enero a mayo, con un pico en marzo (890,00 €) y mucha variación de un mes a otro (tabla de SQL-8). Los pedidos pasan de 3 al mes en enero y febrero a 5 o 6 desde marzo (SQL-2). Con volúmenes tan bajos, los porcentajes de variación engañan, así que se enseñan junto a las cifras absolutas.

Pregunta 2 · El canal más valioso

Definición elegida: ingresos por cliente comprador en el periodo, como aproximación a su valor de vida (customer lifetime value), completada con el porcentaje de clientes que repiten. Se cuenta por persona, no por cuenta.

```
WITH persona AS (
  SELECT customer_id, MIN(customer_id) OVER (PARTITION BY email) AS
persona_id
  FROM customers
),
pedidos_ok AS (
  SELECT pe.persona_id, o.order_id, SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
```

```
  JOIN persona pe     ON pe.customer_id = o.customer_id
  WHERE o.status = 'delivered'
  GROUP BY pe.persona_id, o.order_id
),
por_cliente AS (
  SELECT persona_id, COUNT(*) AS pedidos, SUM(total) AS ingresos
  FROM pedidos_ok
  GROUP BY persona_id
)
SELECT c.channel,
       COUNT(DISTINCT c.customer_id)                    AS
clientes_registrados,
       COUNT(pc.persona_id)                             AS
clientes_compradores,
       ROUND(SUM(pc.ingresos), 2)                       AS ingresos,
       ROUND(SUM(pc.ingresos) / COUNT(pc.persona_id), 2) AS
ingresos_por_comprador,
       ROUND(100.0 * COUNT(*) FILTER (WHERE pc.pedidos >= 2)
             / COUNT(pc.persona_id))                    AS pct_repiten
FROM customers c
LEFT JOIN por_cliente pc ON pc.persona_id = c.customer_id
WHERE c.customer_id IN (SELECT DISTINCT persona_id FROM persona)
GROUP BY c.channel
ORDER BY ingresos_por_comprador DESC;
```

canal	registrados	compradores	ingresos (€)	por comprador (€)	repiten
organic	3	3	1.060,00	353,33	100 %
ads	5	4	1.107,50	276,88	50 %
referral	3	3	625,00	208,33	67 %

El hallazgo clave: en SQL-6, ads tenía el ticket medio (average order value, AOV) más alto (138,44 €). Pero por cliente gana organic, porque todos sus clientes repiten. El ticket medio no es el valor del cliente: quien se queda en el ticket se equivoca de canal.

Pregunta 3 · Calidad de datos (data quality)

- **Cuenta duplicada:** Ana García tiene dos cuentas con el mismo email; se unificaron en una persona.
- **Pedidos no cobrados:** 2 cancelados y 1 devuelto, excluidos de los ingresos. Incluirlos habría sumado 390 € de más.
- **Ciudades:** mayúsculas y espacios inconsistentes y un valor vacío. Se normalizaron; no afectan a esta pregunta.

- **Cliente sin compras:** Hugo Martín, de ads, se registró pero no compró. Cuenta para la conversión de registro a compra (80 % en ads, 100 % en los otros canales).
- **Descuentos (discounts):** se usa el precio cobrado, no el de catálogo.

Pregunta 4 · Recomendación

Conclusión en una frase: los clientes orgánicos son los más valiosos (353 € por cliente y todos repiten), pero con solo 10 compradores y sin el coste de cada canal no se puede decidir todavía dónde invertir.

Recomendación: no recortar ads, que es el canal que más clientes trae, y probar acciones para que esos clientes repitan, como el email de recordatorio de SQL-10. A la vez, reforzar lo orgánico y las recomendaciones con poco coste.

Datos que pediría antes de mover presupuesto: el coste de adquisición (customer acquisition cost, CAC) por canal, sin el que no se puede calcular la rentabilidad; el margen por producto; y más meses o más clientes, porque con 10 personas ninguna diferencia es estadísticamente fiable.

Cómo se evalúa un caso así

Criterio	Qué buscan
Cifras correctas	Definiciones claras: qué es venta, qué es cliente, qué es valioso
Calidad de datos	Que detectes los problemas sin que te los señalen y digas cómo los trataste
Comunicación	Conclusión en la primera frase y una página que se lee en dos minutos
Honestidad	Limitaciones dichas claramente, sin vender más certeza de la que hay
Accionable	Una recomendación concreta y el siguiente paso